STRIPE PROJECT

3. NoSQL Data System

D8.3. NoSQL Queries Examples

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D8.3 : NoSQL Queries / v09 --- 2026/11/05.*

- D8.3.1. Contexte
- D8.3.2. Collection `event_logs`
  - D8.3.2.1. Erreurs récentes d'un service
  - D8.3.2.2. Reconstitution du parcours d'une transaction précise
- D8.3.3. Collection `user_sessions`
  - D8.3.3.1. Historique de sessions d'un client
  - D8.3.3.2. Retrouver la session ayant mené à une transaction précise
- D8.3.4. Collection `ml_features`
  - D8.3.4.1. Lecture d'inférence en temps réel (chemin critique de la collection)
  - D8.3.4.2. Clients à score de fraude élevé pour un marchand (alerting)
- D8.3.5. Collection `customer_feedback`
  - D8.3.5.1. Derniers avis d'un marchand
  - D8.3.5.2. Recherche plein texte sur les retours clients
- D8.3.6. Collection `merchant_config`
  - D8.3.6.1. Lecture du cache avant décision de scoring
  - D8.3.6.2. Vérification de l'activation d'une fonctionnalité
- Références

D8.3.1. Contexte

Exemples de requêtes sur les collections *MongoDB* [D4]. Chaque requête est motivée par un scénario d'accès et un index bien identifiés dans [D4] (§D4.4 et §D4.6, respectivement).

Les requêtes sont exprimées en *Python*, exécutée par un objet `db`, instancié par la classe `MongoClient` du module `pymongo` :

```python
from pymongo import MongoClient

client = MongoClient("mongodb://...")   # connexion au serveur
db = client["stripe"]                   # accès client à la base "stripe"
```

`db` est un objet d'accès à la base `"stripe"`. La chaîne complète d'accès aux documents est : **client → base (`db`) → collection → documents**. Les expressions `db.ml_features` ou `db["ml_features"]` renvoient un proxy de la collection `ml_features` via lequel on peut exécuter des opérations de recherche et d'agrégation comme `find()`, `find_one()` ou `aggregate()`.

> Rappel : ci-dessous, dans les expressions de type `x.sort('a', -1)`, la valeur d'argument `-1` (respectivement, `1`) indique de trier la collection `x` dans l'ordre décroissant *descendant* (respectivement, dans l'ordre *croissant*) des valeurs du champ `a`.

D8.3.2. Collection `event_logs`

D8.3.2.1. Erreurs récentes d'un service

```python
db.event_logs.find({
    "service": "payment-gateway",
    "level": "error",
    "timestamp": {"$gte": datetime(2026, 10, 13, 0, 0, 0)}
}).sort("timestamp", -1).limit(50)
```

Sert l'index `{ level: 1, timestamp: -1 }` - le motif d'accès dominant de la collection .

D8.3.2.2. Reconstitution du parcours d'une transaction précise

```
db.event_logs.find({
    "transaction_id": "b7c2..."
}).sort("timestamp", 1)
```

Sert l'index *sparse* `{ transaction_id: 1 }` (la propriété "sparse" de l'index prend en compte le fait que la majorité des logs n'ont pas le champ `transaction_id`).

D8.3.3. Collection `user_sessions`

D8.3.3.1. Historique de sessions d'un client

```
db.user_sessions.find({
    "customer_id": "c91d..."
}).sort("started_at", -1).limit(20)
```

Sert l'index `{ customer_id: 1, started_at: -1 }` (cf. §D4.6).

D8.3.3.2. Retrouver la session ayant mené à une transaction précise

```
db.user_sessions.find_one({
    "events.metadata.transaction_id": "b7c2..."
})
```

Sert l'index multikey `{ "events.metadata.transaction_id": 1 }` - nécessaire parce que `transaction_id` est imbriqué dans le tableau `events`.

D8.3.4. Collection `ml_features`

D8.3.4.1. Lecture d'inférence en temps réel (chemin critique de la collection)

```
db.ml_features.find_one(
    {"_id": "cust:c91d..."}
)
```

Seule requête qui compte vraiment pour cette collection (cf. §D4.5.4) : une lecture par clé primaire, sans jointure ni agrégation (c'est ce qui permet à la décision de fraude de s'exécuter quelques millisecondes).

D8.3.4.2. Clients à score de fraude élevé pour un marchand (alerting)

```
db.ml_features.find({
    "merchant_id": "a3f1...",
    "model_scores.fraud_v3": {"$gte": 0.8}
}).sort("model_scores.fraud_v3", -1)
```

Sert l'index `{ "model_scores.fraud_v3": -1 }` (cf. §D4.6).

D8.3.5. Collection `customer_feedback`

D8.3.5.1. Derniers avis d'un marchand

```
db.customer_feedback.find({
    "merchant_id": "a3f1..."
}).sort("submitted_at", -1).limit(20)
```

Sert l'index `{ merchant_id: 1, submitted_at: -1 }` (cf. §D4.6).

D8.3.5.2. Recherche plein texte sur les retours clients

```
db.customer_feedback.find({
    "merchant_id": "a3f1...",
    "$text": {"$search": "confirmation lente"}
})
```

Sert l'index *texte* sur `content.text` (cf. §D4.6).

D8.3.6. Collection `merchant_config`

D8.3.6.1. Lecture du cache avant décision de scoring

```
db.merchant_config.find_one(
    { "_id": "a3f1..." }
)
```

L'unique motif d'accès de la collection `merchant_config` est un accès par clé primaire seule. La lecture d'un cache aussi rapide qu'un accès direct.

D8.3.6.2. Vérification de l'activation d'une fonctionnalité

```
db.merchant_config.find_one(
    {"_id": "a3f1..."},
    {"enabled_features": 1, "_id": 0}
)
```

Même motif que ci-dessus, avec projection pour ne renvoyer que le champ utile — pertinent quand cette vérification a lieu à haute fréquence sur le chemin de décision applicatif.

Références

Spécifications

- [D0] Stripe Project Introduction

Livrables

- [D4] NoSQL Data Model